

Robust Scaling

1. Robust Scaling

2. Definition

Robust scaling is a feature scaling technique that transforms numerical data using statistics that are resistant to outliers, namely the *median* and the *interquartile range* (IQR). Unlike standard scaling, which relies on the mean and standard deviation, robust scaling is much less affected by extreme values.

For a feature X , the robust scaled version of a value x is defined by

$$x_{\text{scaled}} = \frac{x - Q_2}{\text{IQR}},$$

where

$$Q_2 = \text{median}, \quad \text{IQR} = Q_3 - Q_1,$$

and Q_1 , Q_2 , Q_3 denote respectively the 25th, 50th, and 75th percentiles.

3. Why Use Robust Scaling?

Robust scaling is particularly useful when the dataset contains outliers. Its main advantages are:

Resistance to outliers: the median and IQR are much less influenced by extreme observations than the mean and standard deviation;

Better representation of the bulk of the data: the central part of the distribution is scaled more appropriately;

No need to remove outliers beforehand: the method can be applied directly;

Practical relevance: many real-world datasets contain measurement errors, rare events, or naturally extreme values.

4. Interpretation of Quartiles

Quartiles divide a sorted dataset into four parts:

- Q_1 : first quartile, or 25th percentile;
- Q_2 : second quartile, i.e. the median;
- Q_3 : third quartile, or 75th percentile;
- $\text{IQR} = Q_3 - Q_1$: range containing the middle 50% of the data.

With robust scaling:

- the median is mapped to 0;
- values near the center of the distribution remain in a moderate range;
- extreme observations remain far from 0, so outliers are still visible.

5. Comparison with Standard Scaling

Standard scaling uses

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma},$$

where μ is the mean and σ the standard deviation. Since both are sensitive to extreme values, the presence of outliers can distort the scaling substantially.

Robust scaling replaces these quantities with the median and the IQR:

$$x_{\text{scaled}} = \frac{x - Q_2}{Q_3 - Q_1}.$$

Hence, it is often more appropriate for skewed or heavy-tailed data.

6. Important Practical Issues

Some edge cases must be handled carefully:

Zero IQR: if $Q_1 = Q_3$, then the denominator is zero and the formula is undefined;

Small datasets: quartile estimates may be unstable when the sample size is very small;

Train-test split: quartiles must be computed on the training set only, then reused on the test set to avoid data leakage;

Column-wise scaling: in a multivariate dataset, robust scaling is applied independently to each feature.

7. When to Use Robust Scaling

Robust scaling is well suited for: financial data, sensor measurements, medical datasets, web analytics, survey data, any machine learning pipeline where outliers are present or suspected.

It is less suitable when the data is already clean and Gaussian-like, or when a bounded range such as $[0, 1]$ is required.

8. Worked Example

Consider the data

$$[10, 20, 30, 40, 50, 60, 70, 80, 90, 1000],$$

where 1000 is an obvious outlier.

The quartiles are

$$Q_1 = 27.5, \quad Q_2 = 55, \quad Q_3 = 82.5.$$

Thus,

$$\text{IQR} = Q_3 - Q_1 = 82.5 - 27.5 = 55.$$

Applying the formula:

$$\frac{10 - 55}{55} = -0.82, \quad \frac{50 - 55}{55} = -0.09, \quad \frac{1000 - 55}{55} = 17.18.$$

We see that the outlier remains clearly extreme, while the bulk of the data is scaled around zero.

9. Python Code

Below is a simple implementation of robust scaling in Python:

```
import statistics

def robust_scaling(values):
    if not values:
        return []

    median_val = statistics.median(values)
    sorted_vals = sorted(values)
    n = len(sorted_vals)

    if n == 1:
        return [0]

    if n % 2 == 0:
        q1 = statistics.median(sorted_vals[:n//2])
        q3 = statistics.median(sorted_vals[n//2:])
    else:
        q1 = statistics.median(sorted_vals[:n//2])
        q3 = statistics.median(sorted_vals[n//2 + 1:])

    iqr = q3 - q1
    if iqr != 0:
        return [(x - median_val) / iqr for x in values]
    else:
        return [x - median_val for x in values]
```

This implementation is simple and effective for educational purposes. In practice, libraries such as `scikit-learn` provide optimized and more flexible implementations of robust scaling.